LAB 4

SOFTWARE ENGINEERING

Name: Yellapantula Venkata Pranav

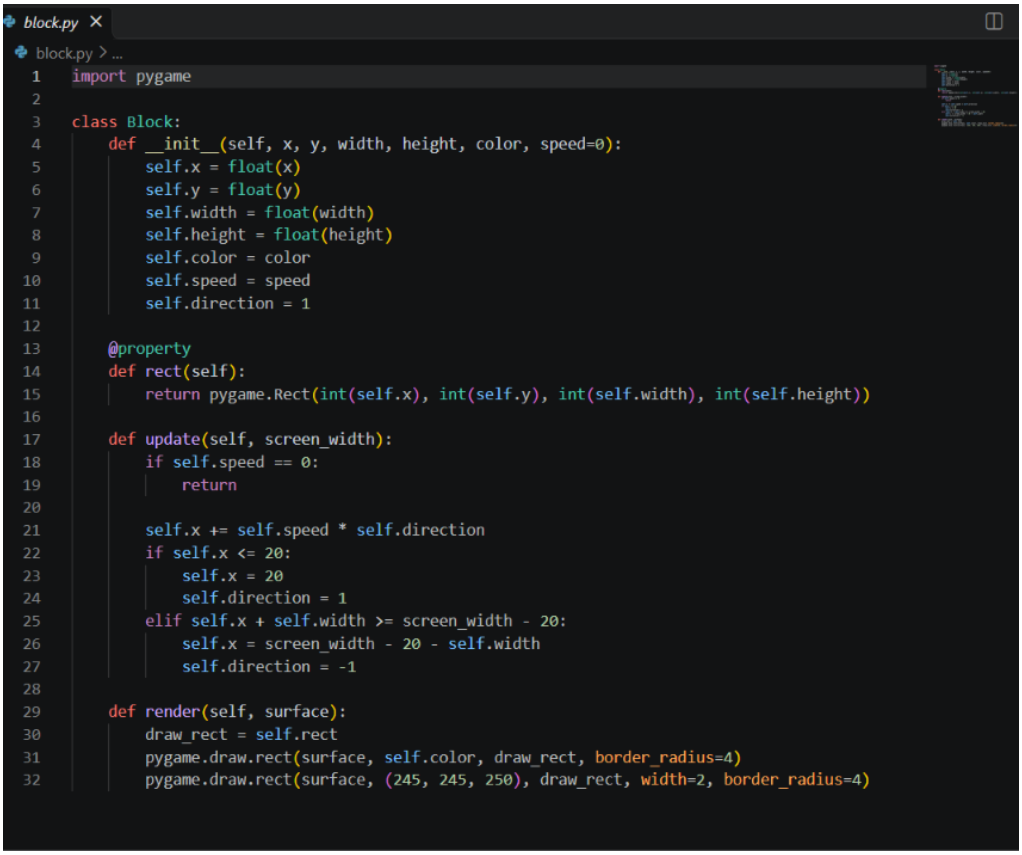
SRN: PES2UG24CS616

Class: 5J

Github link: https://github.com/pranavyv/54_skyscrapper_stack.git

LLM link: https://claude.ai/share/ed80b498-98ea-4c44-8be9-4cecd90b175b

block.py



```python
import pygame

class Block:
    def __init__(self, x, y, width, height, color, speed=0):
        self.x = float(x)
        self.y = float(y)
        self.width = float(width)
        self.height = float(height)
        self.color = color
        self.speed = speed
        self.direction = 1

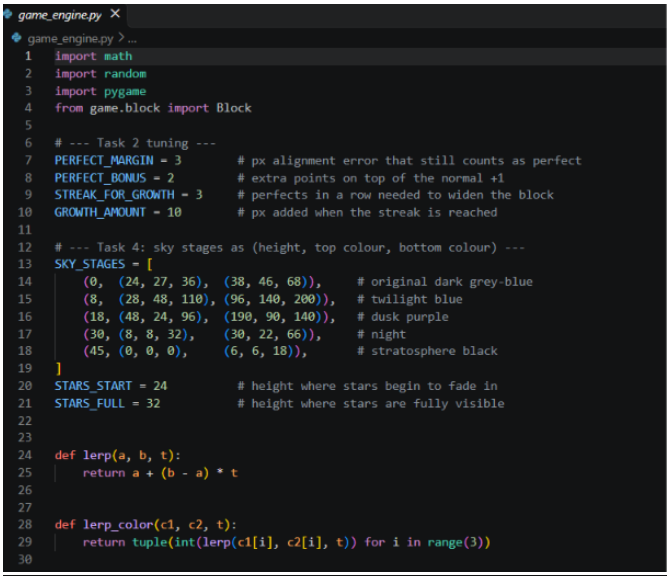
    @property
    def rect(self):
        return pygame.Rect(int(self.x), int(self.y), int(self.width), int(self.height))

    def update(self, screen_width):
        if self.speed == 0:
            return

        self.x += self.speed * self.direction
        if self.x <= 20:
            self.x = 20
            self.direction = 1
        elif self.x + self.width >= screen_width - 20:
            self.x = screen_width - 20 - self.width
            self.direction = -1

    def render(self, surface):
        draw_rect = self.rect
        pygame.draw.rect(surface, self.color, draw_rect, border_radius=4)
        pygame.draw.rect(surface, (245, 245, 250), draw_rect, width=2, border_radius=4)
```

game_engine.py



```python
import math
import random
import pygame
from game.block import Block

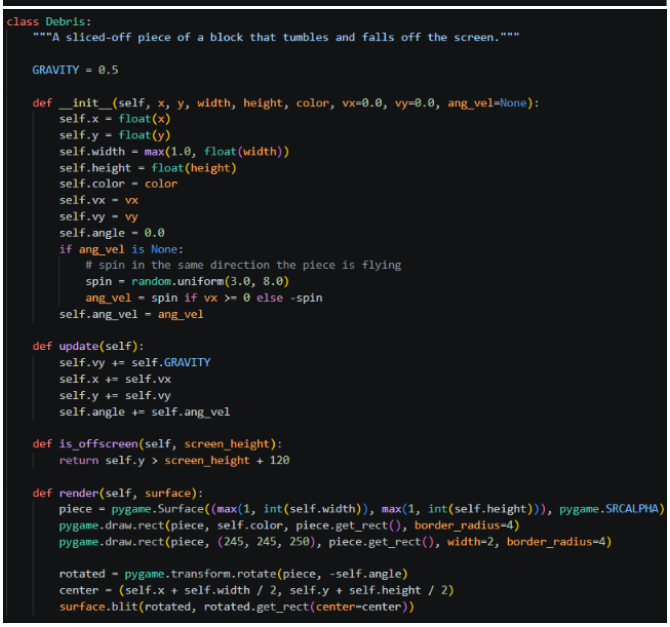
# --- Task 2 tuning ---
PERFECT_MARGIN = 3        # px alignment error that still counts as perfect
PERFECT_BONUS = 2         # extra points on top of the normal +1
STREAK_FOR_GROWTH = 3     # perfects in a row needed to widen the block
GROWTH_AMOUNT = 10        # px added when the streak is reached

# --- Task 4: sky stages as (height, top colour, bottom colour) ---
SKY_STAGES = [
    (0,  (24, 27, 36),  (38, 46, 68)),     # original dark grey-blue
    (8,  (28, 48, 110), (96, 140, 200)),   # twilight blue
    (18, (48, 24, 96),  (190, 90, 140)),   # dusk purple
    (30, (8, 8, 32),    (30, 22, 66)),     # night
    (45, (0, 0, 0),     (6, 6, 18)),       # stratosphere black
]
STARS_START = 24          # height where stars begin to fade in
STARS_FULL = 32           # height where stars are fully visible


def lerp(a, b, t):
    return a + (b - a) * t


def lerp_color(c1, c2, t):
    return tuple(int(lerp(c1[i], c2[i], t)) for i in range(3))

```



```python
class Debris:
    """A sliced-off piece of a block that tumbles and falls off the screen."""

    GRAVITY = 0.5

    def __init__(self, x, y, width, height, color, vx=0.0, vy=0.0, ang_vel=None):
        self.x = float(x)
        self.y = float(y)
        self.width = max(1.0, float(width))
        self.height = float(height)
        self.color = color
        self.vx = vx
        self.vy = vy
        self.angle = 0.0
        if ang_vel is None:
            # spin in the same direction the piece is flying
            spin = random.uniform(3.0, 8.0)
            ang_vel = spin if vx >= 0 else -spin
        self.ang_vel = ang_vel

    def update(self):
        self.vy += self.GRAVITY
        self.x += self.vx
        self.y += self.vy
        self.angle += self.ang_vel

    def is_offscreen(self, screen_height):
        return self.y > screen_height + 120

    def render(self, surface):
        piece = pygame.Surface((max(1, int(self.width)), max(1, int(self.height))), pygame.SRCALPHA)
        pygame.draw.rect(piece, self.color, piece.get_rect(), border_radius=4)
        pygame.draw.rect(piece, (245, 245, 250), piece.get_rect(), width=2, border_radius=4)

        rotated = pygame.transform.rotate(piece, -self.angle)
        center = (self.x + self.width / 2, self.y + self.height / 2)
        surface.blit(rotated, rotated.get_rect(center=center))
```

```python
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.block_height = 28
        self.base_width = 180

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 28)
        self.font_big = pygame.font.SysFont(None, 46)
        self.font_popup = pygame.font.SysFont(None, 40)

        # fixed star field (x, y, radius, twinkle phase), reused across restarts
        self.stars = [
            (random.randint(0, width), random.randint(0, height),
             random.choice([1, 1, 2]), random.uniform(0, math.tau))
            for _ in range(70)
        ]

        self.reset()

    def get_color(self, index):
        palette = [
            (230, 75, 75),   # Crimson
            (240, 140, 45),  # Orange
            (245, 210, 50),  # Gold
            (60, 195, 110),  # Green
            (50, 150, 240),  # Blue
            (165, 80, 225),  # Purple
        ]
        return palette[index % len(palette)]
```

```python
    def reset(self):
        self.score = 0
        self.game_over = False
        self.perfect_streak = 0
        self.debris = []
        self.popups = []
        self.sky_height = 0.0   # eased copy of score so the sky shifts smoothly

        base_x = (self.width - self.base_width) // 2
        base_y = self.height - 60
        base_block = Block(base_x, base_y, self.base_width, self.block_height, self.get_color(0), speed=0)
        self.stack = [base_block]

        self.spawn_active_block()

    def spawn_active_block(self):
        top_block = self.stack[-1]
        next_y = top_block.y - self.block_height - 4
        speed = min(10.0, 4.5 + (len(self.stack) * 0.35))
        color = self.get_color(len(self.stack))

        start_x = 25 if random.choice([True, False]) else self.width - 25 - top_block.width
        self.active_block = Block(start_x, next_y, top_block.width, self.block_height, color, speed=speed)

    # ------------------------------------------------------------
    # Popups & debris helpers
    # ------------------------------------------------------------
    def add_popup(self, text, x, y, color=(255, 215, 0)):
        self.popups.append({"text": text, "x": x, "y": y, "life": 50, "max_life": 50, "color": color})

    def spawn_debris(self, x, y, width, color, vx):
        if width <= 0.5:
            return
        self.debris.append(Debris(x, y, width, self.block_height, color,
                                  vx=vx, vy=random.uniform(-2.0, 0.0)))
```

```
# ------------------------------------------------------------
# Core gameplay
# ------------------------------------------------------------
def drop_block(self):
    if self.game_over:
        return

    top_block = self.stack[-1]
    act = self.active_block

    left = max(act.x, top_block.x)
    right = min(act.x + act.width, top_block.x + top_block.width)
    overlap = right - left

    # Task 1 fix: positive overlap means the block landed on the tower.
    is_successful_drop = overlap > 0

    if not is_successful_drop:
        # Missed completely: the whole block tumbles away.
        self.spawn_debris(act.x, act.y, act.width, act.color, vx=act.direction * 2.0)
        self.game_over = True
        return

    is_perfect = abs(act.x - top_block.x) <= PERFECT_MARGIN
    popup_x = self.width // 2
    popup_y = act.y - 10

    if is_perfect:
        # Task 2: snap into alignment, no trimming, bonus points.
        new_x = top_block.x
        new_w = top_block.width
        self.perfect_streak += 1
        self.score += 1 + PERFECT_BONUS
        self.add_popup("PERFECT!", popup_x, popup_y)
        if self.perfect_streak >= STREAK_FOR_GROWTH:
            grown_w = min(float(self.base_width), new_w + GROWTH_AMOUNT)
            if grown_w > new_w:
                new_x -= (grown_w - new_w) / 2
                new_w = grown_w
                # keep the block inside the arena walls used by Block.update
                new_x = max(20.0, min(new_x, self.width - 20 - new_w))
                self.add_popup("WIDTH UP!", popup_x, popup_y - 34, color=(120, 255, 160))
            self.perfect_streak = 0
    else:
        # Normal drop: trim to the overlap and shed the overhang as debris.
        new_x = left
        new_w = max(10.0, overlap)
        self.perfect_streak = 0
        self.score += 1

        if act.x < left:
            self.spawn_debris(act.x, act.y, left - act.x, act.color, vx=-random.uniform(1.0, 2.5))
        if act.x + act.width > right:
            self.spawn_debris(right, act.y, (act.x + act.width) - right, act.color,
                              vx=random.uniform(1.0, 2.5))

    new_block = Block(new_x, act.y, new_w, self.block_height, act.color, speed=0)
    self.stack.append(new_block)

    if new_block.y < 180:
        shift_amount = self.block_height + 4
        for b in self.stack:
            b.y += shift_amount
        for d in self.debris:
            d.y += shift_amount
        for p in self.popups:
            p["y"] += shift_amount

    self.spawn_active_block()
```

```
def handle_event(self, event):
    if self.game_over:
        if (event.type == pygame.KEYDOWN and event.key in (pygame.K_r, pygame.K_SPACE)) or \
           (event.type == pygame.MOUSEBUTTONDOWN and event.button == 1):
            self.reset()
        return

    if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
        self.drop_block()
    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        self.drop_block()

def update(self):
    if not self.game_over:
        self.active_block.update(self.width)

    # debris and popups keep animating even after game over
    for d in self.debris:
        d.update()
    self.debris = [d for d in self.debris if not d.is_offscreen(self.height)]

    for p in self.popups:
        p["y"] -= 0.8
        p["life"] -= 1
    self.popups = [p for p in self.popups if p["life"] > 0]

    # ease the sky toward the current height
    self.sky_height += (self.score - self.sky_height) * 0.05
```

```
# ---------------------------------------------------------------------
# Rendering
# ---------------------------------------------------------------------
def sky_colors(self):
    h = self.sky_height
    for i in range(len(SKY_STAGES) - 1):
        h0, top0, bot0 = SKY_STAGES[i]
        h1, top1, bot1 = SKY_STAGES[i + 1]
        if h < h1:
            t = max(0.0, (h - h0) / (h1 - h0))
            return lerp_color(top0, top1, t), lerp_color(bot0, bot1, t)
    return SKY_STAGES[-1][1], SKY_STAGES[-1][2]

def draw_sky(self, screen):
    top, bottom = self.sky_colors()
    band = 4
    for y in range(0, self.height, band):
        color = lerp_color(top, bottom, y / self.height)
        pygame.draw.rect(screen, color, (0, y, self.width, band))

    star_strength = (self.sky_height - STARS_START) / (STARS_FULL - STARS_START)
    star_strength = max(0.0, min(1.0, star_strength))
    if star_strength > 0:
        now = pygame.time.get_ticks() / 400.0
        for sx, sy, r, phase in self.stars:
            twinkle = 0.6 + 0.4 * math.sin(now + phase)
            c = int(255 * star_strength * twinkle)
            if c > 10:
                pygame.draw.circle(screen, (c, c, c), (sx, sy), r)
```

```python
def render(self, screen):
    self.draw_sky(screen)

    title_surf = self.font_title.render("Skyscraper Stack", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 16))

    score_surf = self.font_hud.render(f"Height: {self.score}", True, (255, 220, 80))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 54))

    streak_surf = self.font_hud.render(
        f"Perfect streak: {self.perfect_streak}/{STREAK_FOR_GROWTH}", True, (200, 200, 210))
    screen.blit(streak_surf, (12, self.height - 28))

    for b in self.stack:
        b.render(screen)

    for d in self.debris:
        d.render(screen)

    if not self.game_over:
        self.active_block.render(screen)

    for p in self.popups:
        surf = self.font_popup.render(p["text"], True, p["color"])
        surf.set_alpha(int(255 * p["life"] / p["max_life"]))
        screen.blit(surf, (p["x"] - surf.get_width() // 2, int(p["y"])))

    if self.game_over:
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 195))
        screen.blit(overlay, (0, 0))

        over_surf = self.font_big.render("TOWER COLLAPSED!", True, (240, 75, 75))
        screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 40))

        final_surf = self.font_hud.render(f"Final Height: {self.score}", True, (255, 255, 255))
        screen.blit(final_surf, (self.width // 2 - final_surf.get_width() // 2, self.height // 2 + 10))

        restart_surf = self.font_hud.render("Press [Space] or [R] to Play Again", True, (200, 200, 200))
        screen.blit(restart_surf, (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 50))
```

```python
def render(self, screen):
    self.draw_sky(screen)

    title_surf = self.font_title.render("Skyscraper Stack", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 16))

    score_surf = self.font_hud.render(f"Height: {self.score}", True, (255, 220, 80))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 54))

    streak_surf = self.font_hud.render(
        f"Perfect streak: {self.perfect_streak}/{STREAK_FOR_GROWTH}", True, (200, 200, 210))
    screen.blit(streak_surf, (12, self.height - 28))

    for b in self.stack:
        b.render(screen)

    for d in self.debris:
        d.render(screen)

    if not self.game_over:
        self.active_block.render(screen)

    for p in self.popups:
        surf = self.font_popup.render(p["text"], True, p["color"])
        surf.set_alpha(int(255 * p["life"] / p["max_life"]))
        screen.blit(surf, (p["x"] - surf.get_width() // 2, int(p["y"])))

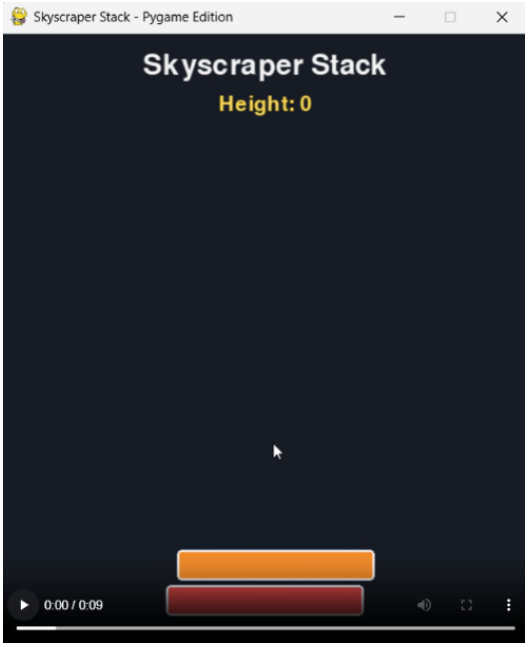
    if self.game_over:
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 195))
        screen.blit(overlay, (0, 0))

        over_surf = self.font_big.render("TOWER COLLAPSED!", True, (240, 75, 75))
        screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 40))

        final_surf = self.font_hud.render(f"Final Height: {self.score}", True, (255, 255, 255))
        screen.blit(final_surf, (self.width // 2 - final_surf.get_width() // 2, self.height // 2 + 10))

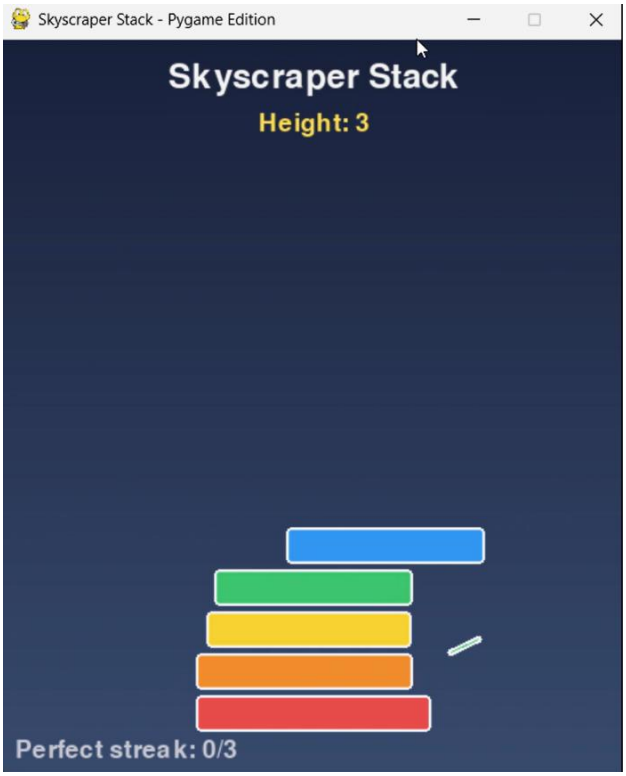
        restart_surf = self.font_hud.render("Press [Space] or [R] to Play Again", True, (200, 200, 200))
        screen.blit(restart_surf, (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 50))
```

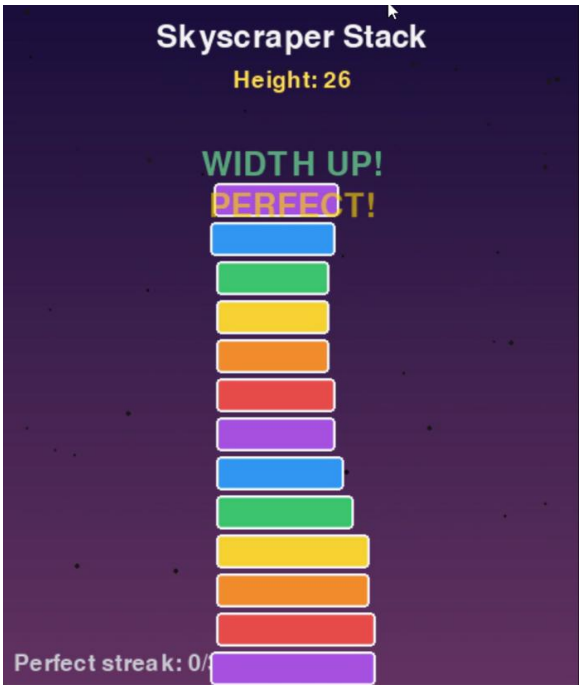
Before:

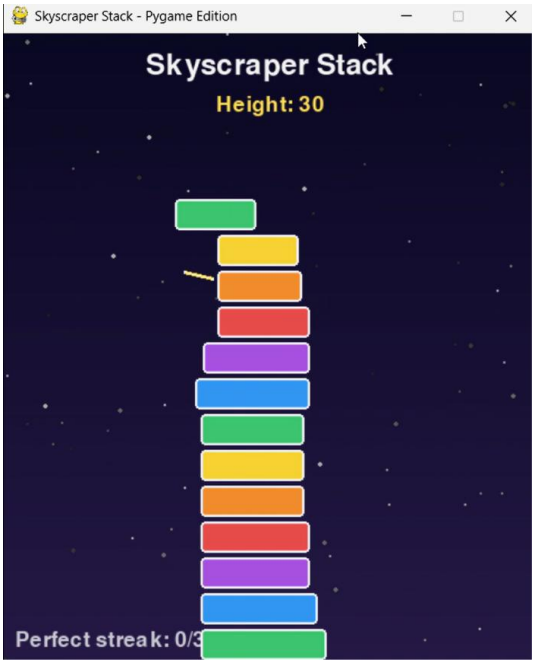

Skyscraper Stack - Pygame Edition
Skyscraper Stack
Height: 0
0:00 / 0:09


Skyscraper Stack - Pygame Edition
Skyscraper Stack
Height: 0
TOWER COLLAPSED!
Final Height: 0
Press [Space] or [R] to Play Again
0:02 / 0:09

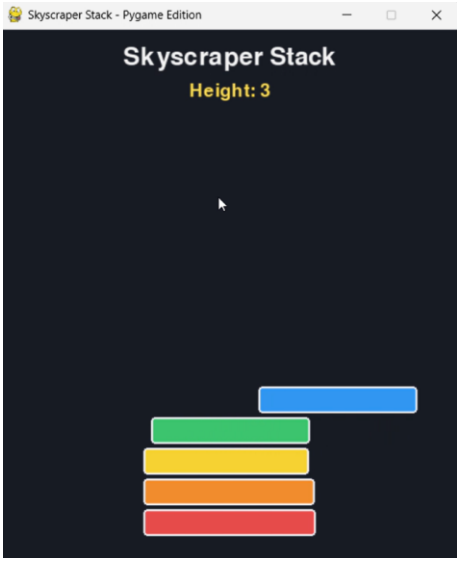
After:


Skyscraper Stack - Pygame Edition
Skyscraper Stack
Height: 3
Perfect streak: 0/3

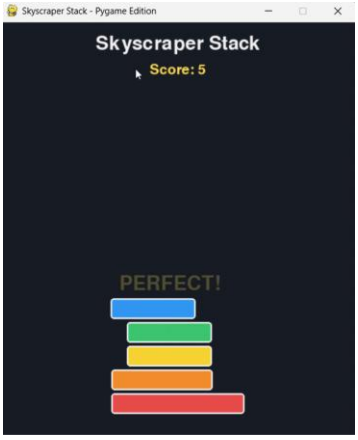

Skyscraper Stack
Height: 26
WIDTH UP!
PERFECT!


Skyscraper Stack - Pygame Edition
Skyscraper Stack
Height: 30
Perfect streak: 0/3

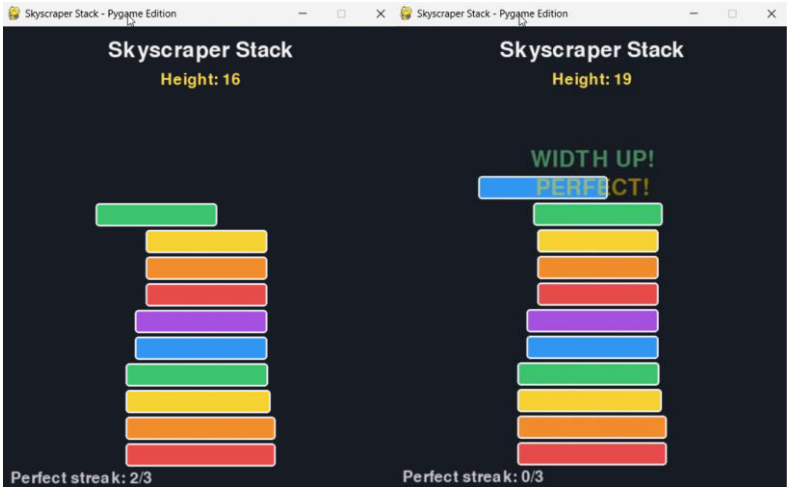
Feature 1:


Skyscraper Stack - Pygame Edition
Skyscraper Stack
Height: 3

Feature 2:



Feature 3:



Feature 4:

